Normalización Avanzada y Redundancia Proyecto integrador de referencia: Food Store

1. Presentación del trabajo

Este trabajo práctico exige aplicar, sobre una situación nueva del dominio Food Store, las dos herramientas centrales de la Unidad 4: llevar un esquema hasta la **Forma Normal de Boyce-Codd (FNBC)** cuando una dependencia funcional lo exige, y **desnormalizar de manera deliberada y controlada** cuando una consulta real del sistema lo justifica por rendimiento.

El trabajo se organiza en dos partes independientes entre sí, pero unidas por el mismo criterio profesional: ninguna decisión de diseño —normalizar o desnormalizar— se toma por intuición. Se toma analizando dependencias funcionales con rigor, en la Parte 1, y midiendo con EXPLAIN ANALYZE, en la Parte 2.

2. Objetivos de aprendizaje

Al finalizar este trabajo, el estudiante deberá ser capaz de:

1. Identificar dependencias funcionales a partir de una regla de negocio redactada en lenguaje natural.
2. Calcular clausuras de atributos y determinar el conjunto completo de claves candidatas de un esquema.
3. Determinar, con la definición formal de FNBC, si un esquema cumple o no esa forma normal, y aplicar el algoritmo de descomposición sin pérdida cuando no la cumple.
4. Reconocer, en una consulta real, cuándo el costo de un recorrido o de sucesivas reuniones (JOIN) justifica una desnormalización controlada.
5. Escribir al menos una consulta que explote una estructura desnormalizada, y compararla contra su equivalente normalizada mediante EXPLAIN ANALYZE.

3. Consigna general y modo de trabajo

El trabajo se resuelve sobre una copia de la base Food Store ya poblada. La Parte 1 introduce una extensión puntual del dominio, que el estudiante debe crear y poblar con una instancia de ejemplo suficiente para exhibir el problema. La Parte 2 se resuelve directamente sobre el esquema ya existente del proyecto (tablas pedido, detalle_pedido, producto, categoria).

Toda afirmación técnica debe estar acompañada del artefacto SQL o de la salida de EXPLAIN ANALYZE que la demuestra: no se acepta una respuesta exclusivamente narrativa.

4. Parte 1 — Análisis y aplicación de la FNBC: `ControlLoteAlmacen`

4.1. Contexto de dominio

Food Store extiende su operación hacia la distribución mayorista: incorpora **depósitos** propios y recibe de sus proveedores **lotes** de mercadería, que deben pasar un control de calidad antes de ingresar al inventario. Cuando un lote es de gran volumen, se fracciona y se controla simultáneamente desde más de un depósito. Por cada depósito que interviene en el control de un lote, se designa un **responsable de control** a cargo de esa fracción. La regla de negocio, relevada con el área de logística, es la siguiente:

- Para un lote y un depósito interviniente dados, el responsable de control queda unívocamente determinado.
- Cada responsable de control pertenece, como dato maestro de la dotación de personal, a un único depósito: no controla lotes coordinados desde depósitos distintos.

Este relevamiento se traduce en el siguiente esquema inicial, ya cargado con una instancia de ejemplo. Se asume que las tablas maestras lote y deposito de esta extensión mayorista ya existen (de manera análoga a como sucursal se asumió existente al tratar el caso `AsignacionEntrega`):

```
CREATE TABLE control_lote_almacen (
  lote_id  BIGINT  NOT  NULL  REFERENCES  lote(id),    deposito_id
BIGINT NOT NULL REFERENCES deposito(id),  responsable_control_id
BIGINT  NOT  NULL  REFERENCES  usuario(id),   PRIMARY KEY (lote_id,
deposito_id)
);

INSERT INTO control_lote_almacen VALUES
 (501, 30, 801),
 (502, 30, 801),
 (503, 31, 802);
```

4.2. Se pide

a) Enunciar, en notación formal, las dependencias funcionales que se desprenden de la regla de negocio, usando los nombres de atributo `LoteID`, `DepositoID` y `ResponsableControlID`.

1) Para un lote y un depósito dados, el responsable queda unívocamente determinado.

   $\{ \text{LoteID}, \text{DepositoID} \} \rightarrow \text{ResponsableControlID}.$

   Esta dependencia modela la PK declarada.

2) Para cada responsable hay un único depósito.

   Esta dependencia introduce la dependencia parcial problemática.

   $\text{ResponsableControlID} \rightarrow \text{DepositoID}$

   No existe evidencia de que LoteID → DepositoID ni de que LoteID → ResponsableControlID sean válidas (un lote puede ser controlado desde varios depósitos, y por tanto por varios responsables), así que no se postulan.

b) Calcular la clausura de los subconjuntos de atributos que resulten candidatos razonables y determinar el conjunto **completo** de claves candidatas del esquema. Indicar qué atributos

son primos y cuáles no.

Universo completo de atributos: U={LoteID, DepositoID, ResponsableControlID}

Subconjuntos candidatos razonables

Clausura de {LoteID, DepositoID}:

{LoteID, DepositoID}+ = {LoteID, DepositoID} = {LoteID, DepositoID, ResponsableControlID}

{LoteID, DepositoID} es superclave. ¿Es mínima? Verificar subconjuntos:

- {LoteID}+ = {LoteID} — no cubre U.
- {DepositoID}+ = {DepositoID} — no cubre U.

Ningún subconjunto propio es superclave → {LoteID, DepositoID} es clave candidata.

Clausura de {ResponsableControlID}:

{ResponsableControlID}+ = {ResponsableControlID} → aplico DF2: agrego D

= {ResponsableControlID, DepositoID} → no hay más DF aplicables

{ResponsableControlID}+ = {ResponsableControlID, DepositoID} — no cubre U (falta LoteID).

{ResponsableControlID} no es clave candidata.

Clausura de {LoteID, ResponsableControlID}:

{LoteID, ResponsableControlID}+ = {LoteID, ResponsableControlID} → aplico DF2: agrego DepositoID

= {LoteID, ResponsableControlID, DepositoID} → cubre U ✓

{LoteID, ResponsableControlID} es superclave. ¿Es mínima?

{LoteID}+ = {LoteID} — no cubre.

• {ResponsableControlID}+ = {ResponsableControlID, DepositoID} — no cubre (falta LoteID).

{LoteID, ResponsableControlID} es clave candidata.

No existen otras combinaciones no exploradas que sean minimales.

Conjunto de claves candidatas: { {LoteID, DepositoID}, {LoteID, ResponsableControlID}}

Atributos primos: LoteID, DepositoID, ResponsableControlID

Atributos no primos: No existen atributos no primos, los 3 son primos

c) Determinar, con la definición formal de FNBC (no con la de 3FN), si `control_lote_almacen`

cumple o no la forma normal. Justificar identificando explícitamente la dependencia funcional violatoria, si la hubiera, y explicando por qué su determinante no es superclave.

`control_lote_almacen` no cumple con FNBC porque `ResponsableControlID → DepositoID` es una dependencia funcional cuyo determinante (`ResponsableControlID`) no es una superclave.

d) Con la instancia de ejemplo provista, redactar en un párrafo cada una de las tres anomalías clásicas (inserción, borrado, actualización) que la violación detectada habilita, describiendo un escenario concreto sobre esos datos.

d) Anomalías sobre la instancia de ejemplo

La instancia de referencia:

lote_id	deposito_id	responsable_control_id
501	30	801
502	30	801
503	31	802

Anomalía de inserción:
Imaginemos que se incorpora el responsable 803 y se lo asigna al depósito 32, pero todavía no tiene ningún lote a su cargo. En este caso no se puede guardar la relación entre el responsable y el depósito porque lote_id forma parte de la clave primaria y no admite valores nulos. Esto obliga a esperar a que exista un lote real o a crear un dato ficticio, por lo que la información de que el responsable 803 pertenece al depósito 32 no puede registrarse correctamente desde el principio.

Anomalía de borrado:
Supongamos que el lote 503 es el único que relaciona al responsable 802 con el depósito 31. Si se elimina la fila (503, 31, 802) porque ya no se necesita registrar ese control, también desaparece la única evidencia de que el responsable 802 trabaja en ese depósito. Como consecuencia, al borrar un dato operativo se pierde información importante sobre el personal.

Anomalía de actualización:
El responsable 801 figura en dos registros distintos, correspondientes a los lotes 501 y 502, ambos asociados al depósito 30. Si este responsable es trasladado al depósito 33, será necesario modificar el depósito en las dos filas. Si una de ellas no se actualiza por error, la base de datos quedará inconsistente, ya que el mismo responsable aparecerá asignado a dos depósitos diferentes al mismo tiempo.

e) Aplicar el algoritmo de descomposición sin pérdida visto en clase y entregar el script SQL completo de las tablas resultantes, incluyendo sus claves primarias, sus restricciones `FOREIGN KEY`, y una vista de compatibilidad que reconstruya la relación original mediante una reunión natural.

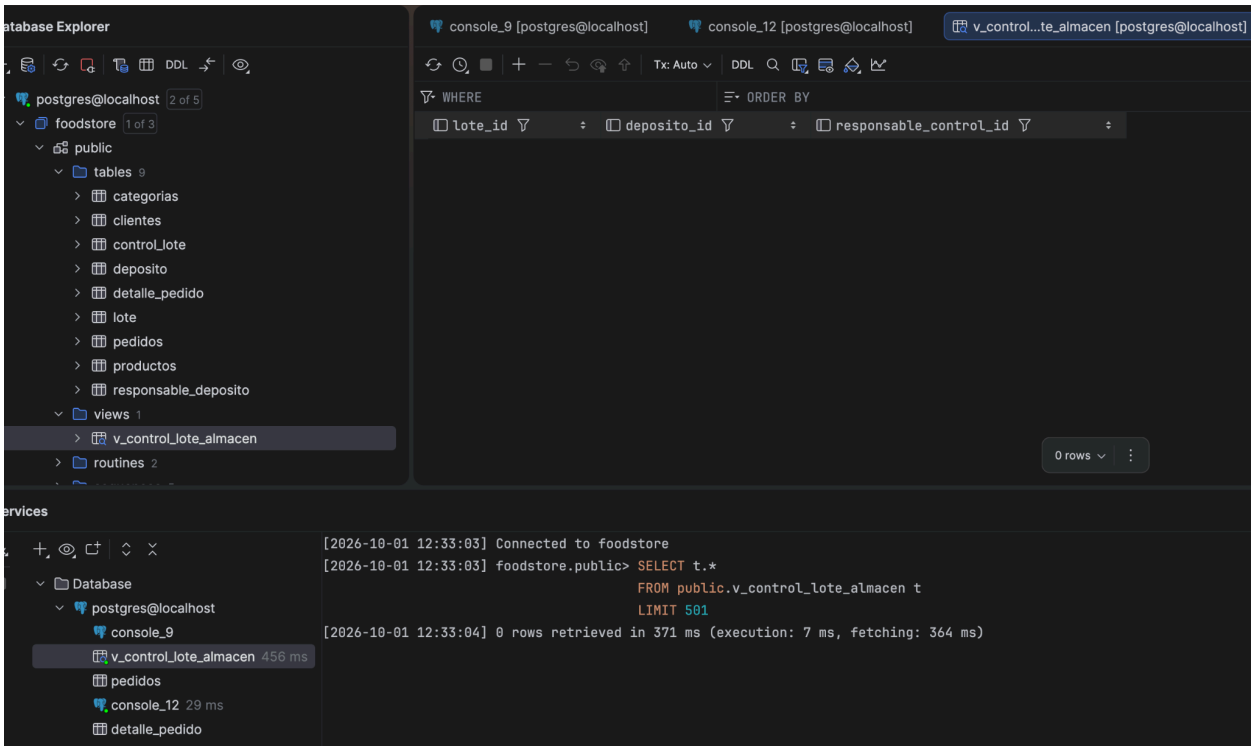
```
-- ============================================================
-- Creacion de tablas
-- ============================================================
CREATE TABLE IF NOT EXISTS lote (
    id BIGINT PRIMARY KEY
);

CREATE TABLE IF NOT EXISTS deposito (
    id BIGINT PRIMARY KEY
);


CREATE TABLE responsable_deposito (
    responsable_control_id BIGINT NOT NULL
    REFERENCES clientes(id),
    deposito_id BIGINT NOT NULL
    REFERENCES deposito(id),
    PRIMARY KEY (responsable_control_id)
);


CREATE TABLE control_lote (
    lote_id BIGINT NOT NULL
    REFERENCES lote(id),
    responsable_control_id BIGINT NOT NULL
    REFERENCES responsable_deposito(responsable_control_id),
    PRIMARY KEY (lote_id, responsable_control_id)
);
-- ============================================================
-- Vista de compatibilidad
-- ============================================================
CREATE OR REPLACE VIEW v_control_lote_almacen AS
SELECT
     cl.lote_id,
     rd.deposito_id,
     cl.responsable_control_id
     FROM control_lote cl
     JOIN responsable_deposito rd
     ON cl.responsable_control_id = rd.responsable_control_id;
```

```
[2026-10-01 12:30:40] foodstore.public> CREATE TABLE IF NOT EXISTS lote (
                                            id BIGINT PRIMARY KEY
                                        )
[2026-10-01 12:30:40] completed in 9 ms
[2026-10-01 12:30:46] foodstore.public> CREATE TABLE IF NOT EXISTS deposito (
                                        id BIGINT PRIMARY KEY
                                        )
[2026-10-01 12:30:46] completed in 3 ms
[2026-10-01 12:31:50] foodstore.public> CREATE TABLE responsable_deposito (
                                        responsable_control_id BIGINT NOT NULL
                                        REFERENCES clientes(id),
                                        deposito_id BIGINT NOT NULL
                                        REFERENCES deposito(id),
                                        PRIMARY KEY (responsable_control_id)
                                        )
[2026-10-01 12:31:50] completed in 9 ms
[2026-10-01 12:32:24] foodstore.public> CREATE TABLE control_lote (
                                        lote_id BIGINT NOT NULL
                                        REFERENCES lote(id),
                                        responsable_control_id BIGINT NOT NULL
                                        REFERENCES responsable_deposito(responsable_control_id),
                                        PRIMARY KEY (lote_id, responsable_control_id)
                                        )
[2026-10-01 12:32:24] completed in 6 ms
```



f) Justificar por escrito, invocando el criterio de superclave sobre el atributo común de la descomposición, por qué la reunión de las dos tablas resultantes es sin pérdida, y migrar

los datos de la instancia de ejemplo hacia las tablas descompuestas, verificando la equivalencia mediante la vista de compatibilidad.

La descomposición se considera sin pérdida porque permite reconstruir la tabla original sin agregar registros incorrectos ni perder información. Según el teorema de Heath, una descomposición $R \rightarrow \{R1,R2\}$ es sin pérdida cuando el atributo que comparten ambas relaciones es una superclave de al menos una de ellas.

En este caso, el atributo común entre R1 (responsable_deposito) y R2 es responsable_control_id. Dentro de R1, este atributo funciona como clave primaria, por lo que identifica de manera única cada registro y se considera una superclave.

Como el atributo compartido cumple esta condición, la unión de ambas relaciones mediante un JOIN sobre responsable_control_id permite recuperar exactamente los datos de la relación original. De esta manera, la descomposición no genera tuplas espurias ni provoca pérdida de información.

Migración de datos:

```
-- Paso 1: poblar las tablas maestras con los IDs referenciados

        INSERT INTO lote (id) VALUES (501), (502), (503);

        INSERT INTO deposito (id) VALUES (30), (31);

-- Los usuarios 801 y 802 se asumen ya existentes en la tabla usuario

-- del esquema food_store. Si no existen, insertar primero:

-- INSERT INTO usuario (nombre, apellido, mail, contrasena)

-- VALUES ('Resp','801','resp801@fs.com','hash'),

-- ('Resp','802','resp802@fs.com','hash');


-- Paso 2: poblar R1 — un registro por responsable

INSERT INTO responsable_deposito (responsable_control_id, deposito_id) VALUES (801, 30), (802, 31);


 -- Paso 3: poblar R2 — un registro por (lote, responsable)

INSERT INTO control_lote (lote_id, responsable_control_id) VALUES (501, 801), (502, 801), (503, 802);


-- Verificación: la vista debe devolver exactamente la instancia original

SELECT * FROM v_control_lote_almacen ORDER BY lote_id;
```

Resultado esperado de la vista:

lote_id	deposito_id	responsable_id
501	30	801
502	30	801
503	31	802

La vista devuelve las mismas tres tuplas que tenía el INSERT original. Esto demuestra que no se perdió información ni se agregaron registros que no correspondían, por lo que la descomposición es sin pérdida.

Efecto sobre las anomalías: con el nuevo esquema, la relación entre un responsable y un depósito se guarda directamente en responsable_deposito. Así, se puede registrar al responsable 803 en el depósito 32 aunque todavía no tenga un lote asignado. Además, eliminar un lote no borra la información del responsable y, si se cambia el depósito de 801, solo es necesario modificar una fila.

5. Parte 2 — Consulta con desnormalización controlada

5.1. Escenario

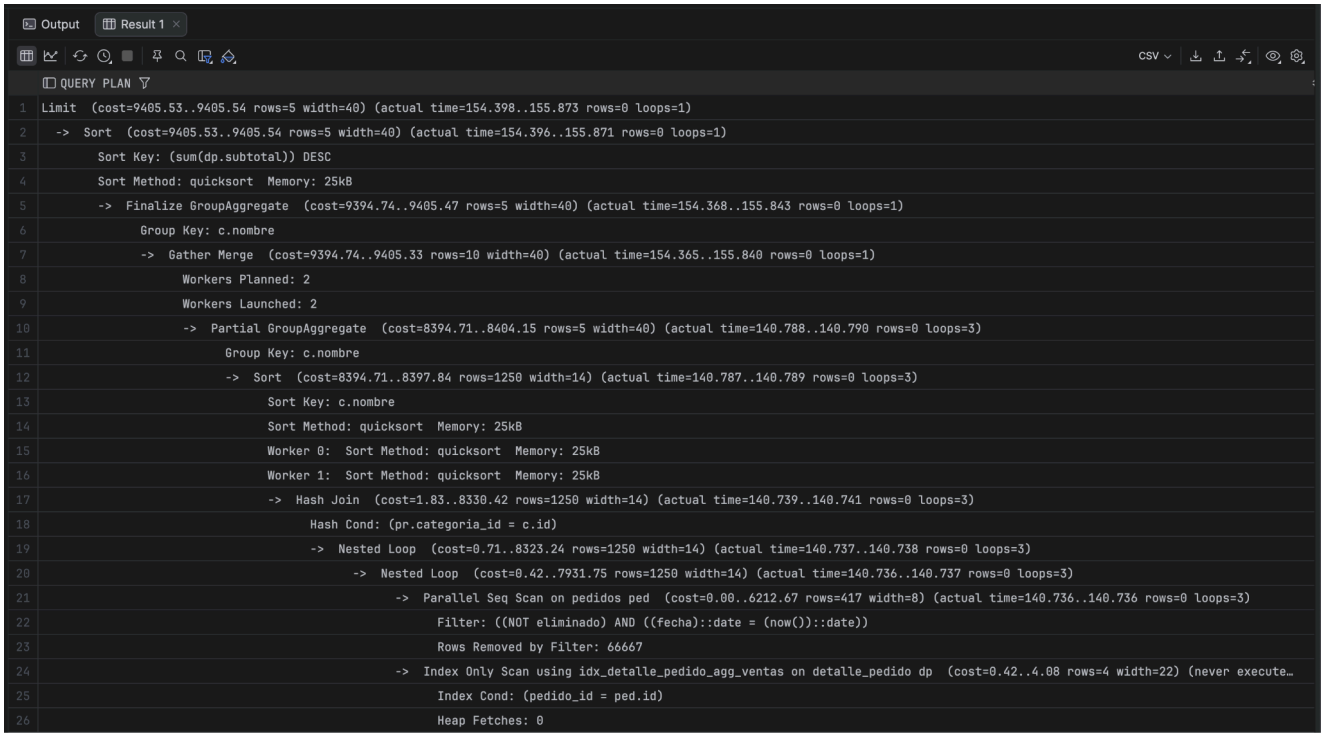
El panel de administración de Food Store necesita mostrar, en tiempo real y con actualización frecuente, el **top 5 de categorías por monto vendido en el día**. La consulta que resuelve este reporte hoy recorre `detalle_pedido`, la une con producto, con `categoria` y con `pedido` para filtrar por fecha y por los registros no eliminados, y agrupa y ordena el resultado:

```
SELECT
    c.nombre AS categoria,
    SUM(dp.subtotal) AS total_vendido
FROM detalle_pedido dp
JOIN productos pr ON pr.id = dp.producto_id
JOIN categorias c ON c.id = pr.categoria_id
JOIN pedidos ped ON ped.id = dp.pedido_id
WHERE ped.fecha::date = NOW()::date
  AND ped.eliminado = FALSE
GROUP BY c.nombre
ORDER BY total_vendido DESC
LIMIT 5;
```

A medida que la base de Food Store crece, esta consulta —ejecutada muchas veces por minuto por el panel— empieza a demandar un tiempo de respuesta notorio.

5.2. Se pide

a) Ejecutar la consulta anterior con EXPLAIN ANALYZE sobre una instancia de Food Store razonablemente poblada, y reportar el tiempo de ejecución y el nodo del plan que domina el costo.



```
QUERY PLAN
Limit  (cost=9405.53..9405.54 rows=5 width=40) (actual time=154.398..155.873 rows=0 loops=1)
  ->  Sort  (cost=9405.53..9405.54 rows=5 width=40) (actual time=154.396..155.871 rows=0 loops=1)
        Sort Key: (sum(dp.subtotal)) DESC
        Sort Method: quicksort  Memory: 25kB
        ->  Finalize GroupAggregate  (cost=9394.74..9405.47 rows=5 width=40) (actual time=154.368..155.843 rows=0 loops=1)
              Group Key: c.nombre
              ->  Gather Merge  (cost=9394.74..9405.33 rows=10 width=40) (actual time=154.365..155.840 rows=0 loops=1)
                    Workers Planned: 2
                    Workers Launched: 2
                    ->  Partial GroupAggregate  (cost=8394.71..8404.15 rows=5 width=40) (actual time=140.788..140.790 rows=0 loops=3)
                          Group Key: c.nombre
                          ->  Sort  (cost=8394.71..8397.84 rows=1250 width=14) (actual time=140.787..140.789 rows=0 loops=3)
                                Sort Key: c.nombre
                                Sort Method: quicksort  Memory: 25kB
                                Worker 0:  Sort Method: quicksort  Memory: 25kB
                                Worker 1:  Sort Method: quicksort  Memory: 25kB
                                ->  Hash Join  (cost=1.83..8330.42 rows=1250 width=14) (actual time=140.739..140.741 rows=0 loops=3)
                                      Hash Cond: (pr.categoria_id = c.id)
                                      ->  Nested Loop  (cost=0.71..8323.24 rows=1250 width=14) (actual time=140.737..140.738 rows=0 loops=3)
                                            ->  Nested Loop  (cost=0.42..7931.75 rows=1250 width=14) (actual time=140.736..140.737 rows=0 loops=3)
                                                  ->  Parallel Seq Scan on pedidos ped  (cost=0.00..6212.67 rows=417 width=8) (actual time=140.736..140.736 rows=0 loops=3)
                                                        Filter: ((NOT eliminado) AND ((fecha)::date = (now())::date))
                                                        Rows Removed by Filter: 66667
                                                  ->  Index Only Scan using idx_detalle_pedido_agg_ventas on detalle_pedido dp  (cost=0.42..4.08 rows=4 width=22) (never execute…
                                                        Index Cond: (pedido_id = ped.id)
                                                        Heap Fetches: 0
                                            ->  Index Scan using productos_pkey on productos pr  (cost=0.29..0.31 rows=1 width=16) (never executed)
                                                  Index Cond: (id = dp.producto_id)
                                      ->  Hash  (cost=1.05..1.05 rows=5 width=16) (never executed)
                                            ->  Seq Scan on categorias c  (cost=0.00..1.05 rows=5 width=16) (never executed)
Planning Time: 3.011 ms
Execution Time: 157.459 ms
```

b) Elegir **un** patrón de desnormalización controlada (columna precalculada con disparador, o vista materializada) y justificar la elección en no más de un párrafo, respondiendo al menos estas tres preguntas: ¿qué evidencia medida en (a) motiva la decisión?, ¿qué mecanismo garantiza que el dato redundante no se desincronice de su fuente de verdad?, y ¿es reversible sin pérdida de información?

Elegimos el patrón de vistas materializadas(MATERIALIZED VIEW) con refresco concurrente (REFRESH MATERIALIZED VIEW CONCURRENTLY). La decisión se debe a que en la evidencia medida con EXPLAIN ANALYZE, la cual muestra que la consulta original es lenta porque obliga al motor a hacer cuatro uniones JOIN y sumar todos los detalles de pedidos cada vez que el panel pide el reporte.

Para evitar que los datos se desincronizan se utiliza la ejecución periódica de la instrucción REFRESH MATERIALIZED VIEW CONCURRENTLY y gracias a un índice único sobre (categoria, fecha), se actualiza la información en segundo plano sin bloquear las lecturas de los usuarios.

Esta decisión es totalmente reversible sin pérdida de información porque las tablas originales se mantienen intactas si eliminamos la vista con un DROP MATERIALIZED VIEW, los datos históricos no se pierden y el reporte simplemente vuelve a ejecutarse sobre el esquema

normalizado.

c) Implementar el patrón elegido con el script SQL completo correspondiente (la estructura desnormalizada y su mecanismo de sincronización).

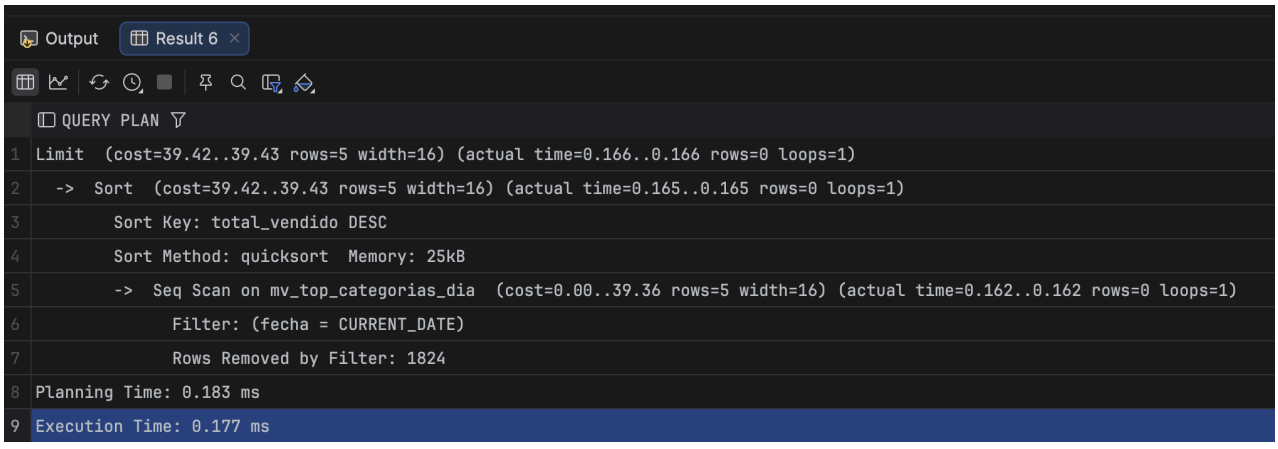
```
CREATE MATERIALIZED VIEW mv_top_categorias_dia
AS SELECT
      c.nombre AS categoria,
      ped.fecha::date AS fecha,
      SUM(dp.subtotal) AS total_vendido
FROM detalle_pedido dp
JOIN productos pr ON pr.id = dp.producto_id
JOIN categorias c ON c.id = pr.categoria_id
JOIN pedidos ped ON ped.id = dp.pedido_id
WHERE ped.eliminado = FALSE
GROUP BY c.nombre, ped.fecha::date;

CREATE UNIQUE INDEX ux_mv_top_cat_dia ON mv_top_categorias_dia
    (categoria, fecha);

REFRESH MATERIALIZED VIEW CONCURRENTLY mv_top_categorias_dia;
```

d) Escribir **al menos una consulta** que resuelva el mismo reporte del punto 5.1 leyendo directamente de la estructura desnormalizada creada en (c), en lugar de recorrer y unir las cuatro tablas originales. Ejecutarla con EXPLAIN ANALYZE y presentar, en una tabla de dos columnas (antes / después), el tiempo de ejecución y el tipo de nodo dominante de cada plan.

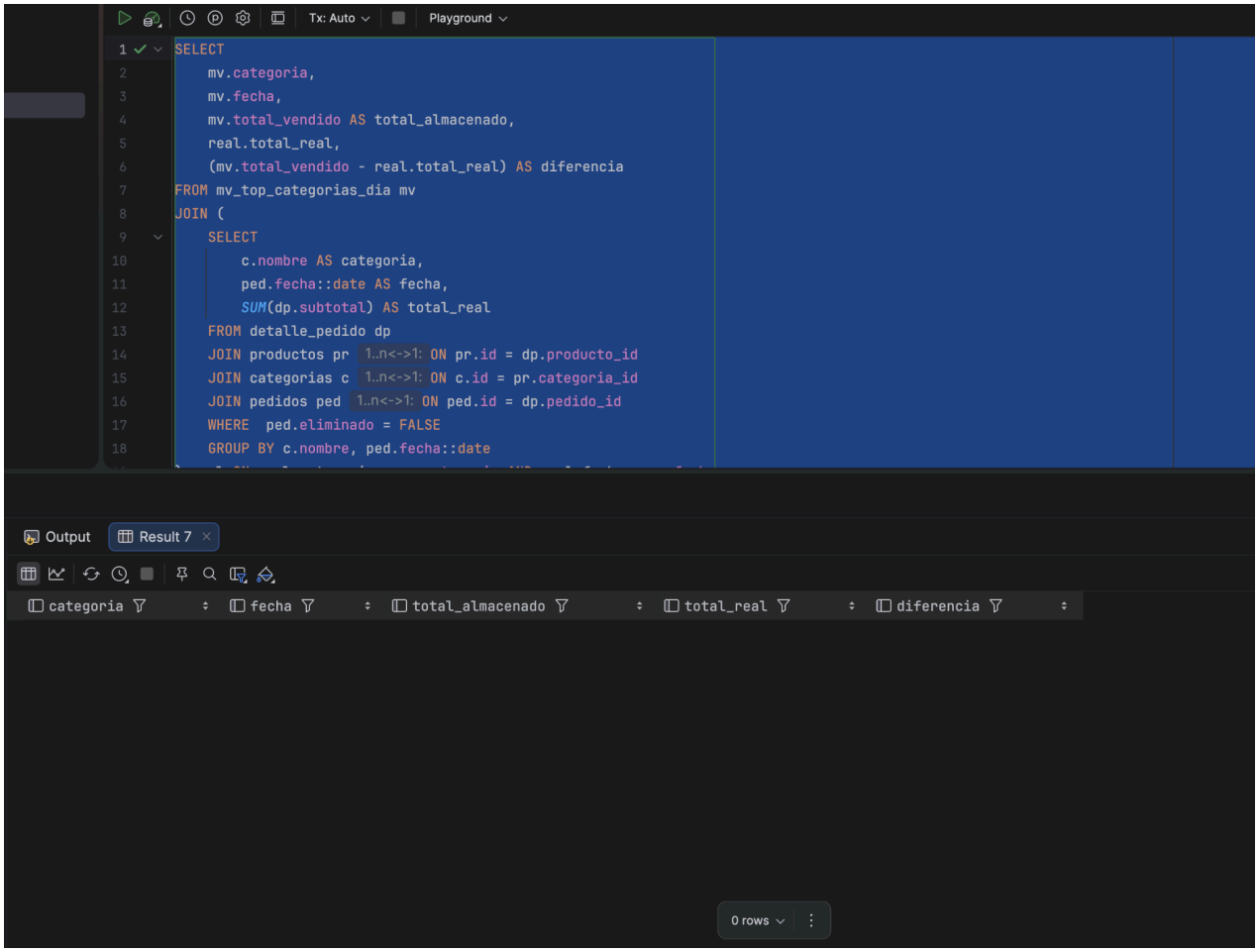
```
EXPLAIN ANALYZE
SELECT
    categoria,
    total_vendido
FROM mv_top_categorias_dia
WHERE fecha = CURRENT_DATE
ORDER BY total_vendido DESC
LIMIT 5;
```



e) Escribir un breve script de auditoría que detecte si el dato redundante introducido en (c) se desincronizó de su fuente de verdad, y ejecutarlo sobre la base ya migrada (su resultado debe ser vacío).

```
SELECT
    mv.categoria,
    mv.fecha,
    mv.total_vendido AS total_almacenado,
    real.total_real,
    (mv.total_vendido - real.total_real) AS diferencia
FROM mv_top_categorias_dia mv
JOIN (
    SELECT
        c.nombre AS categoria,
        ped.fecha::date AS fecha,
        SUM(dp.subtotal) AS total_real
    FROM detalle_pedido dp
    JOIN productos pr ON pr.id = dp.producto_id
    JOIN categorias c ON c.id = pr.categoria_id
    JOIN pedidos ped ON ped.id = dp.pedido_id
    WHERE ped.eliminado = FALSE
    GROUP BY c.nombre, ped.fecha::date
) real ON real.categoria = mv.categoria AND real.fecha =
   mv.fecha


WHERE mv.total_vendido <> real.total_real;
```



6. Entregables

1. `tp_fnbc_control_lote.sql` — esquema original, instancia de ejemplo, tablas descompuestas, vista de compatibilidad y migración de datos verificada (Parte 1).
2. `tp_desnormalizacion_top_categorias.sql` — estructura desnormalizada, su mecanismo de sincronización, la consulta pedida en 5.2(d) y el script de auditoría de 5.2(e) (Parte 2).
3. **Informe breve** (PDF o Word) con: las dependencias funcionales y claves candidatas de la Parte 1, la demostración de violación de FNBC, la justificación de la unión sin pérdida, las capturas de EXPLAIN ANALYZE antes y después de la Parte 2, y la justificación del patrón de desnormalización elegido.7.

Formato de entrega Link con el repositorio de trabajo con los tres artefactos anteriores. El informe y los scripts se evalúan de manera conjunta: una afirmación sin el artefacto que la demuestre no se considera resuelta.